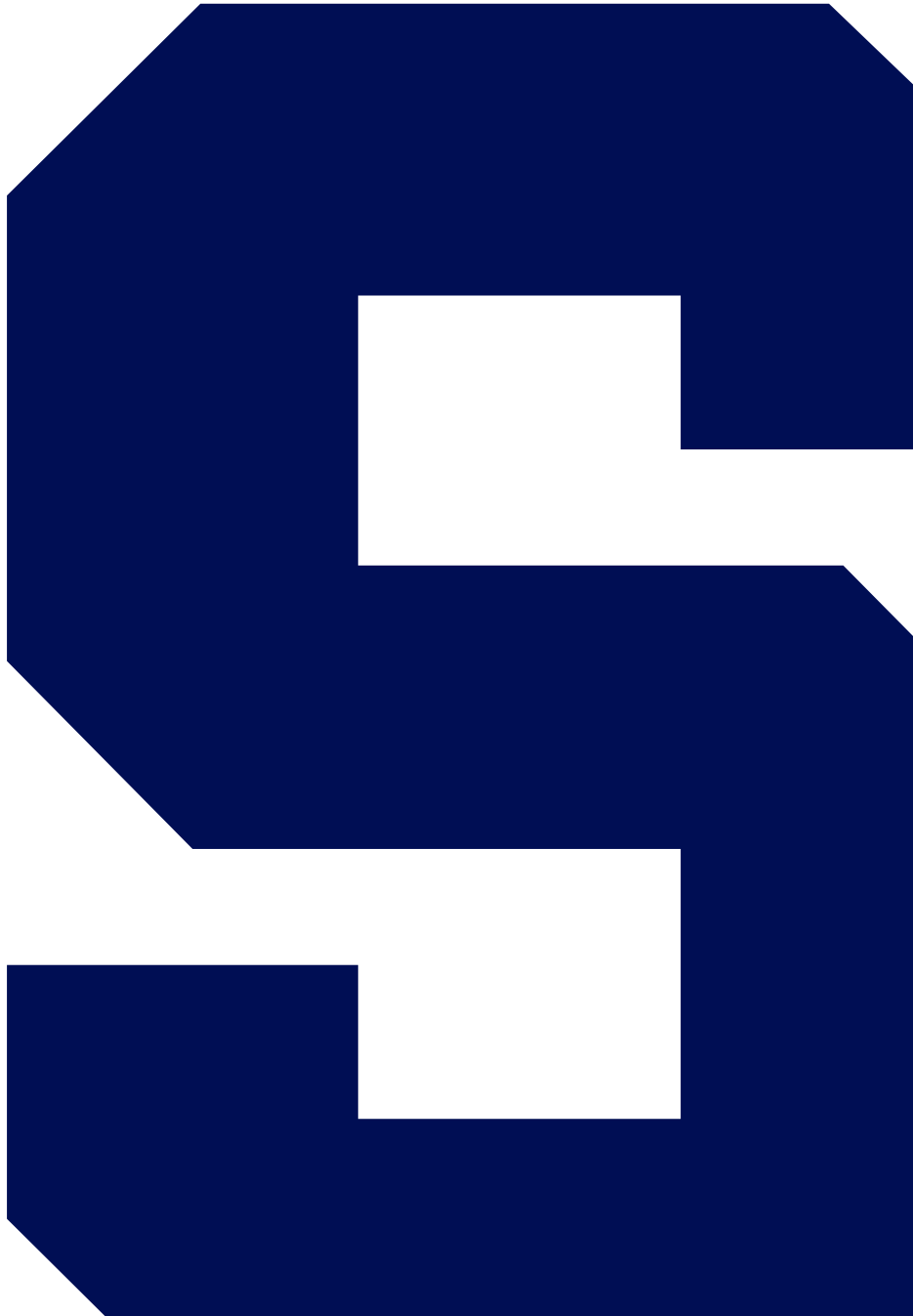
Decision Trees Intro



What is a Decision Tree?

- A **Decision Tree** is a flowchart-like tree structure where each internal node represents a feature (or attribute), each branch represents a decision rule, and each leaf node represents the outcome.
- The decision tree analyzes multiple possible outcomes of a situation and predicts the best possible move.

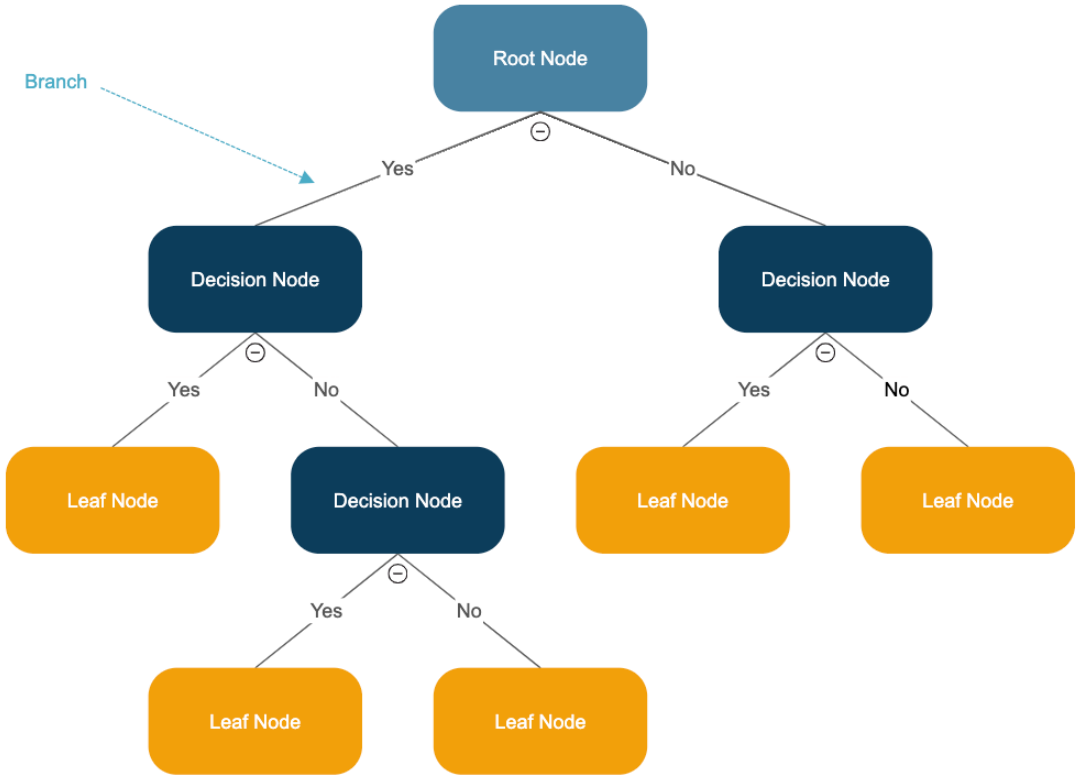


Image source: https://www.smartdraw.com/decision-tree/

How Do Decision Trees Work?

- Divide the dataset into subsets based on different conditions until we make the data as *pure* as possible in terms of the target variable.
- Purity here refers to how mixed the classes in the subsets are.
- The decision tree aims to achieve subsets where the target variable has a single class.
- Targets can be categorical or continuous.

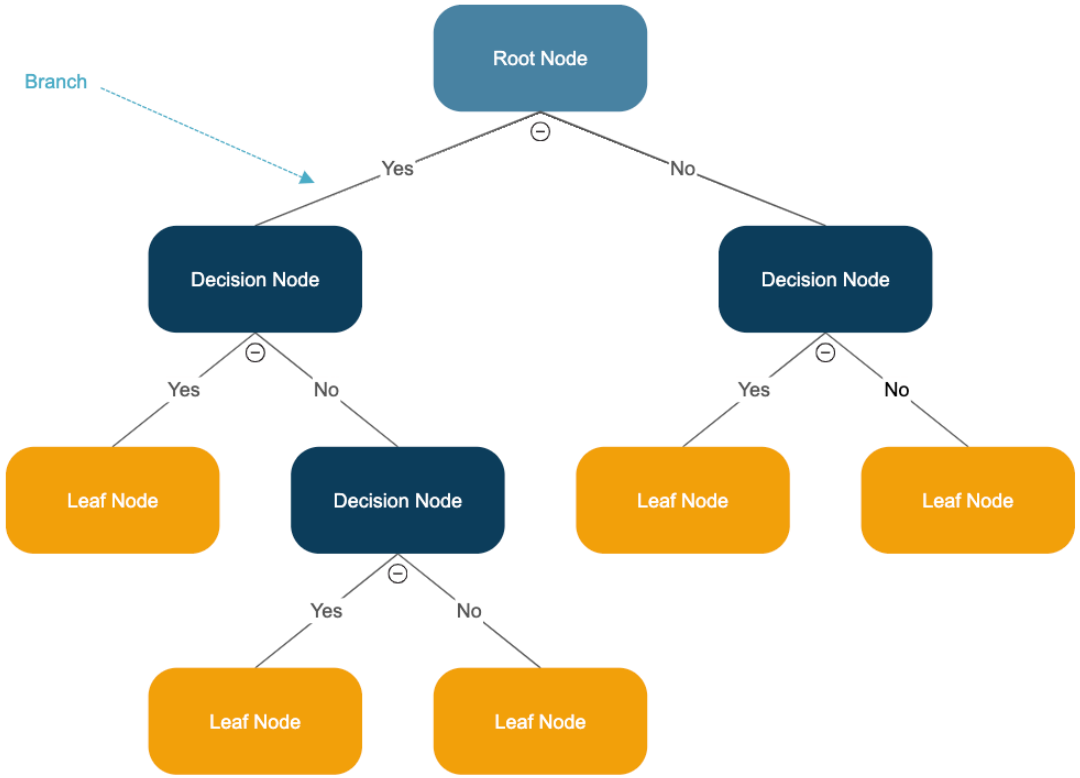


Image source: https://www.smartdraw.com/decision-tree/

Measures Used in Building Trees

- **Gini Impurity**: Measures the probability that a randomly chosen sample from a set would be incorrectly labeled if it was randomly labeled according to the distribution of labels in the set.
- **Entropy**: A measure of the randomness or unpredictability in the dataset. Higher entropy means more mixed labels in a node.
- **Information Gain**: Measures the effectiveness of an attribute in reducing uncertainty.

Advantages and Disadvantages

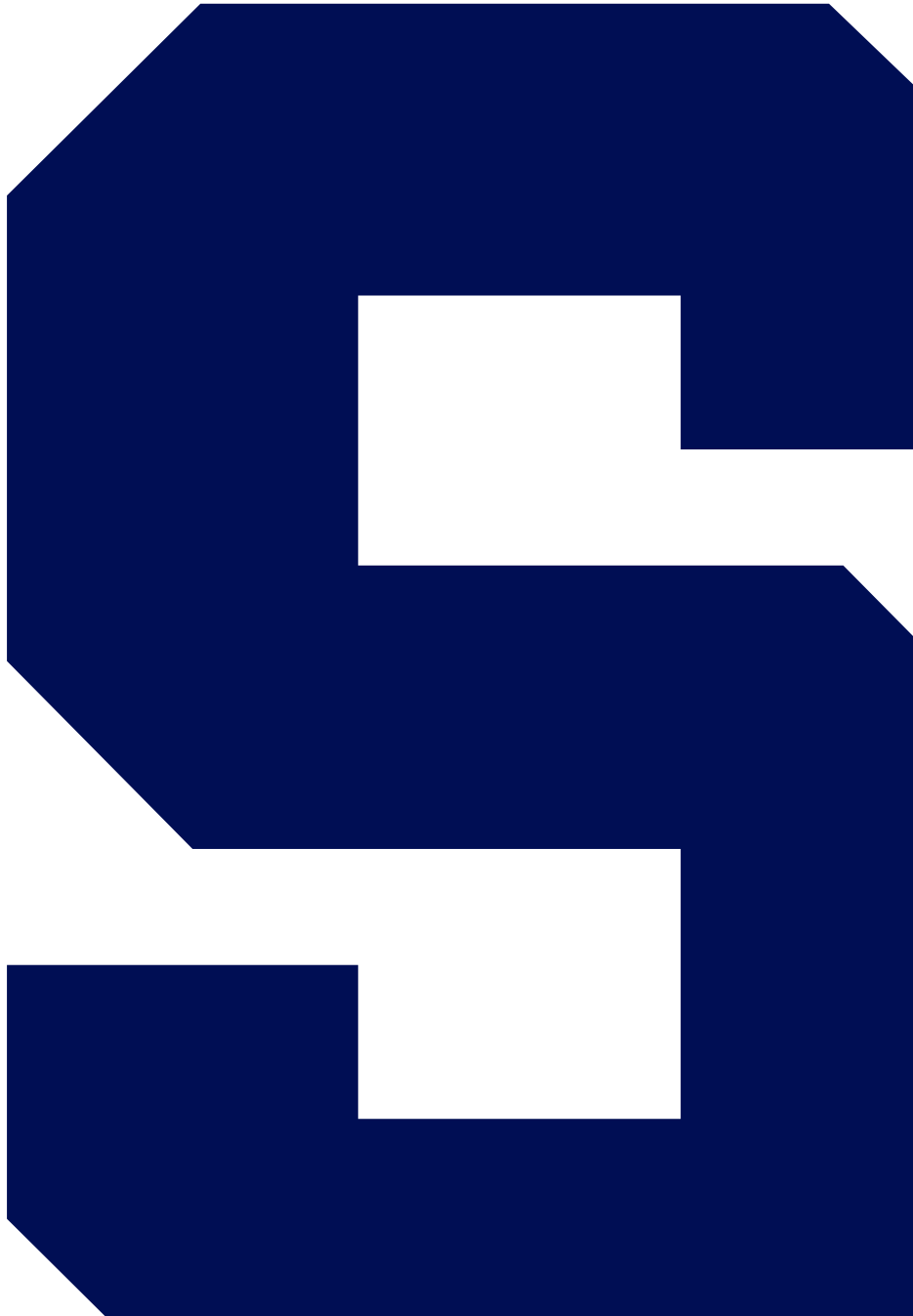
Advantages:

- Simple to understand and interpret.
- Requires less data preprocessing (no need for normalization, dummy variables, etc.).
- Handles both continuous and discrete variables.
- Performs well with large datasets.

Disadvantages:

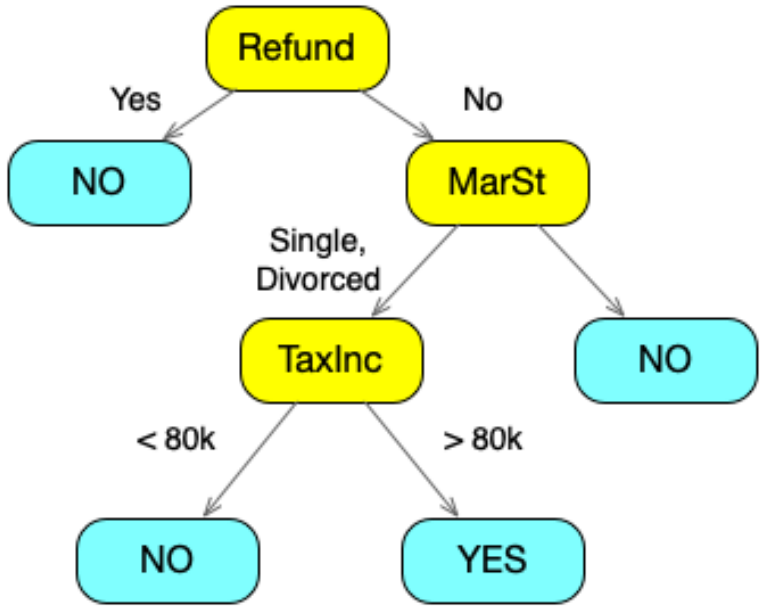
- Sensitive to noisy data, leading to overfitting.
- Biased toward features with more levels.
- Can create biased trees if some classes dominate.

Building a Decision Tree



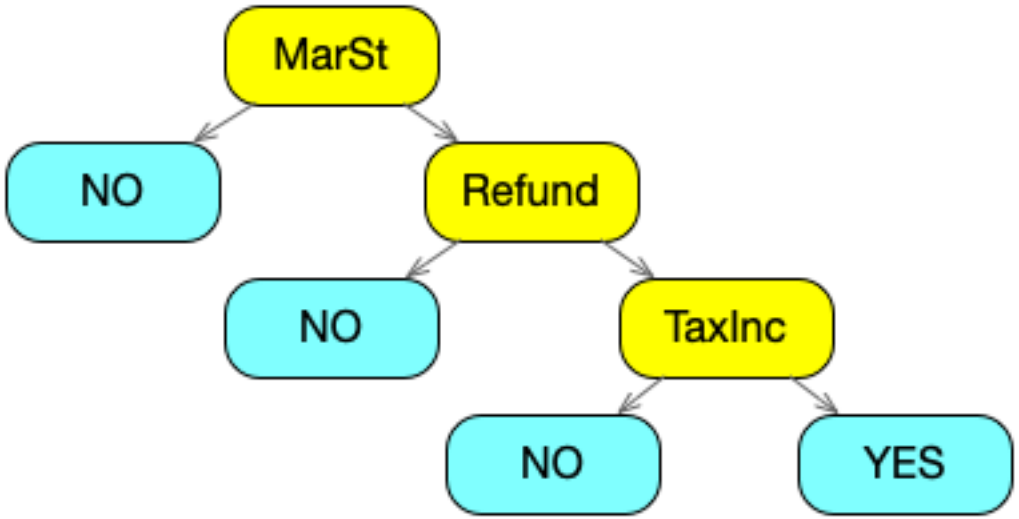
Example Decision Tree

Tid	Refund	Marital Status	Taxable Income (K)	Cheat
1	Yes	Single	125	No
2	No	Married	100	No
3	No	Single	70	No
4	Yes	Married	120	No
5	No	Divorced	95	Yes
6	No	Married	60	No
7	Yes	Divorced	220	No
8	No	Single	85	Yes
9	No	Married	75	No
10	No	Single	90	Yes



An Alternative Decision Tree

Tid	Refund	Marital Status	Taxable Income (K)	Cheat
1	Yes	Single	125	No
2	No	Married	100	No
3	No	Single	70	No
4	Yes	Married	120	No
5	No	Divorced	95	Yes
6	No	Married	60	No
7	Yes	Divorced	220	No
8	No	Single	85	Yes
9	No	Married	75	No
10	No	Single	90	Yes



How Do We Choose Between Possible Splits?

- When building a decision tree, we seek to identify splits that maximize "information gain" - that is, we pick splits that are maximally effective at separating our classes in order of increasing performance.
- To maximize information gain, we can sort features according to their impurity - or in other words, how mixed the feature is.
- The Gini index or Gini coefficient is a metric that quantifies the impurity of a set.

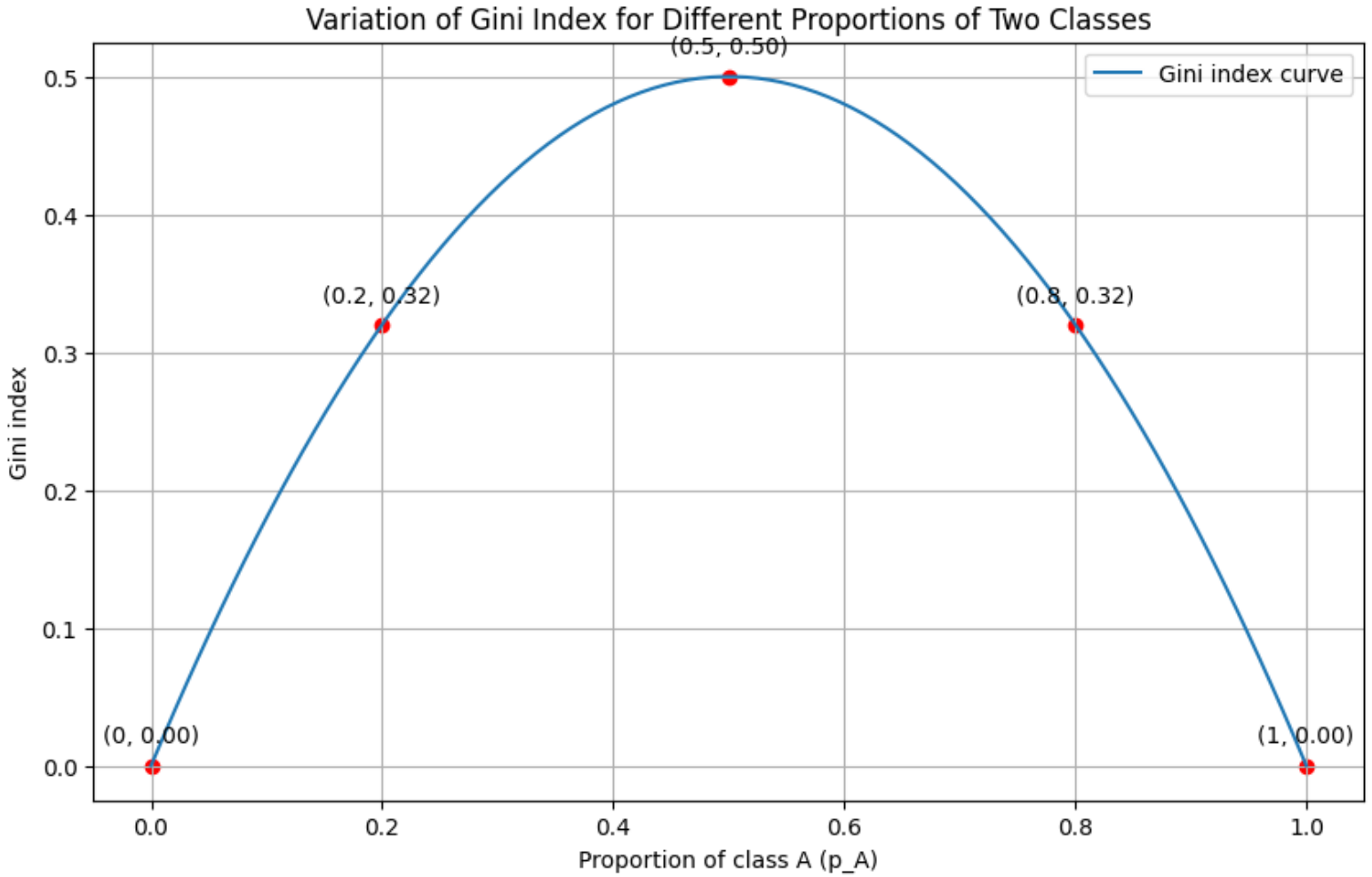
Gini Index

- The Gini index varies between 0 and 1, where 0 expresses the purest form (all elements are of the same class) and 1 implies that the elements are randomly distributed across various classes.

$$Gini(t) = 1 - \sum_{i=1}^{c} (p_i)^2$$

- Here, p_i is the proportion of the samples in the node t that belong to class i, and c is the number of distinct classes.

Gini Index Illustrated



Calculating the Gini Index: Root Node

Tid	Refund	Marital Status	Taxable Income (K)	Cheat
1	Yes	Single	125	No
2	No	Married	100	No
3	No	Single	70	No
4	Yes	Married	120	No
5	No	Divorced	95	Yes
6	No	Married	60	No
7	Yes	Divorced	220	No
8	No	Single	85	Yes
9	No	Married	75	No
10	No	Single	90	Yes

Before determining splits, we calculate the impurity of the entire dataset with respect to the class variable, Cheat.

$$\text{Gini}_{\text{root}} = 1 - ((\frac{7}{10})^2 + (\frac{3}{10})^2) = 1 - (0.49 + 0.09) = 1 - 0.58 = 0.42$$

Calculating the Gini Index: Evaluating Splits

Tid	Refund	Marital Status	Taxable Income (K)	Cheat
1	Yes	Single	125	No
2	No	Married	100	No
3	No	Single	70	No
4	Yes	Married	120	No
5	No	Divorced	95	Yes
6	No	Married	60	No
7	Yes	Divorced	220	No
8	No	Single	85	Yes
9	No	Married	75	No
10	No	Single	90	Yes

To identify the best split, we will calculate the Gini impurity for each potential split of each feature, then select the split that reduces impurity the most. Here, we start with the Refund variable.

$$\text{Gini}_{\text{Refund=Yes}} = 1 - \left(\left(\frac{0}{3}\right)^2 + \left(\frac{3}{3}\right)^2\right) = 1 - (0 + 1) = 0$$

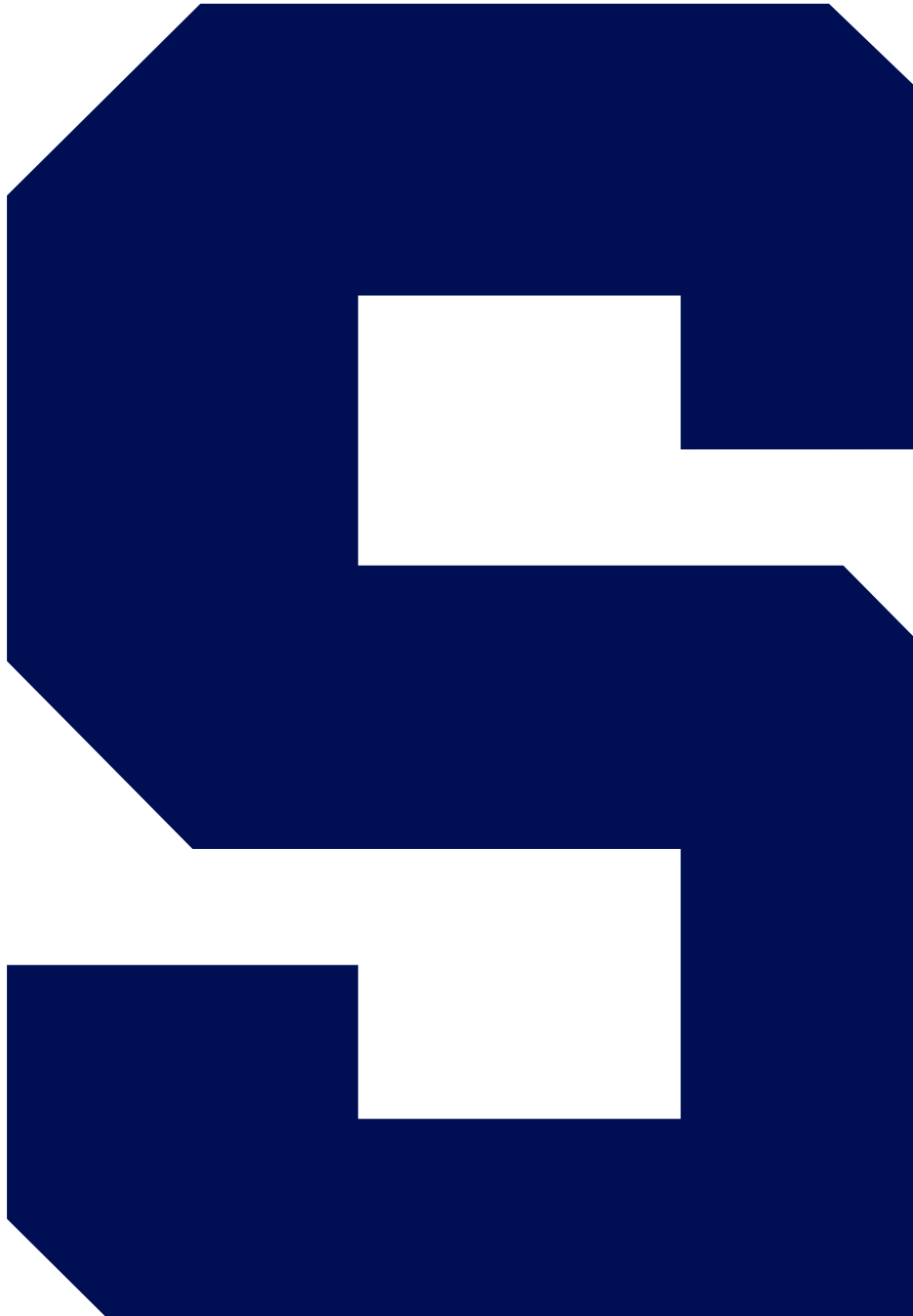
$$\text{Gini}_{\text{Refund=No}} = 1 - \left(\left(\frac{3}{7}\right)^2 + \left(\frac{4}{7}\right)^2\right) = 1 - \left(\frac{9}{49} + \frac{16}{49}\right) = 1 - \frac{25}{49} = \frac{24}{49} \approx 0.49$$

$$\text{Gini}_{\text{Refund}} = \frac{3}{10} \times 0 + \frac{7}{10} \times 0.49 = 0 + 0.343 = 0.343$$

Using Gini to Build a Tree

1. **Calculate Gini for the current node**: Before making a split, calculate the Gini index for the samples currently within the node.
2. **Calculate Gini for each candidate split**: For each candidate feature and split point, divide the dataset into two child nodes. Calculate the Gini index for each child.
3. **Calculate weighted sum of child note indices**: Take a weighted sum of the Gini indices of the child nodes. The weight for each node is the proportion of samples in that node relative to the parent node.
4. **Find the best split**: Choose the feature and split point that results in the smallest weighted sum of child node Gini indices.
5. **Recursion**: Continue this process for each child node (either until nodes contain samples from a single class or some stopping criteria are met).
6. **Evaluate the tree**: After building the tree, evaluate its performance and prune it as necessary, possibly using Gini as one of the criteria for pruning as well.

Ensemble Methods: Wisdom of the Crowd

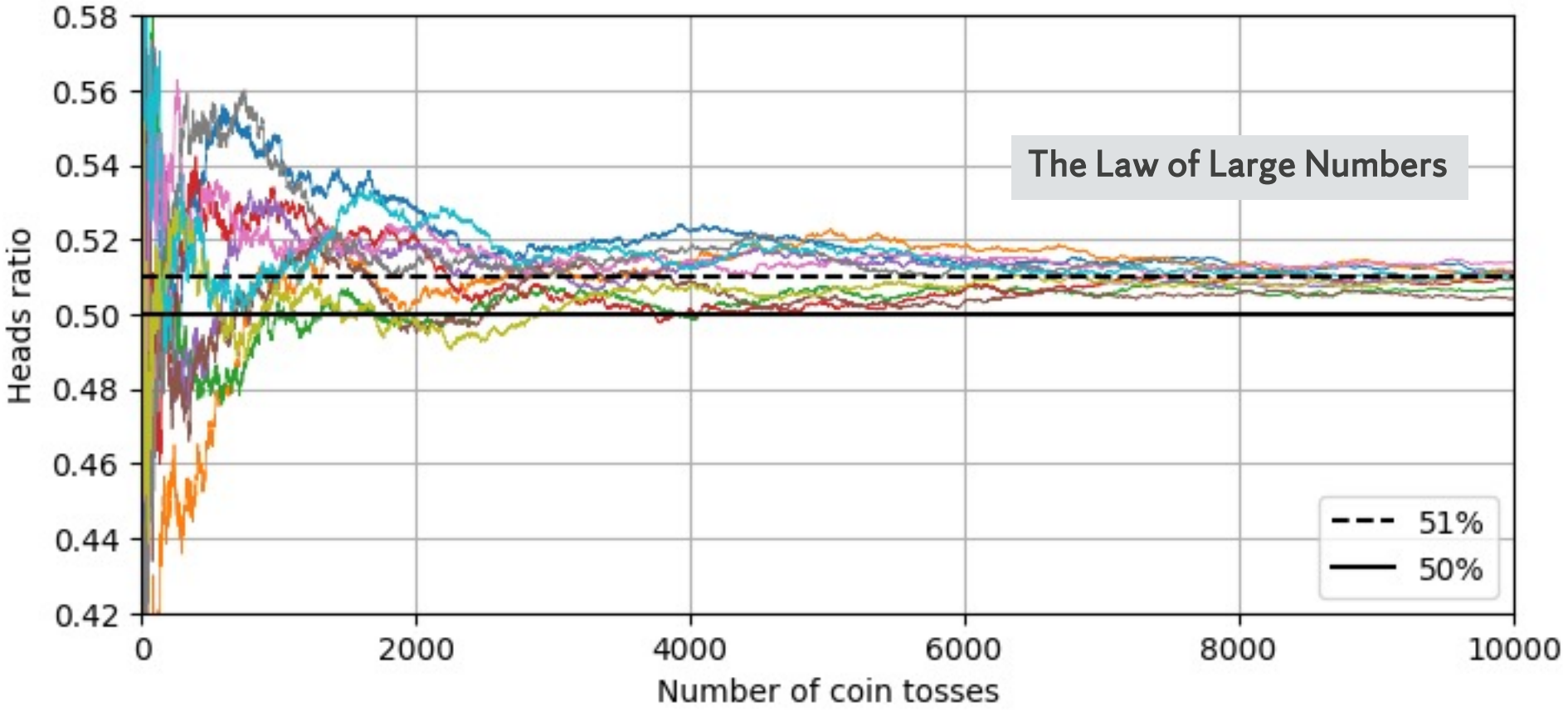


Why Ensemble Methods

- Simple models are often easy to interpret but sometimes lack power and suffer from either high bias (e.g., logistic regression) or high variance (e.g., decision trees).
- One way to address this is gathering many simple models together and aggregate the predictions of these models to achieve a better, aggregate answer.
- This aggregation has been referred to as the "**wisdom of the crowd**" and forms the basis of ensemble methods in machine learning.
- Basic intuition: If different models have strengths and weaknesses and make different errors, then combining them can often **average out their errors** and yield a better prediction.

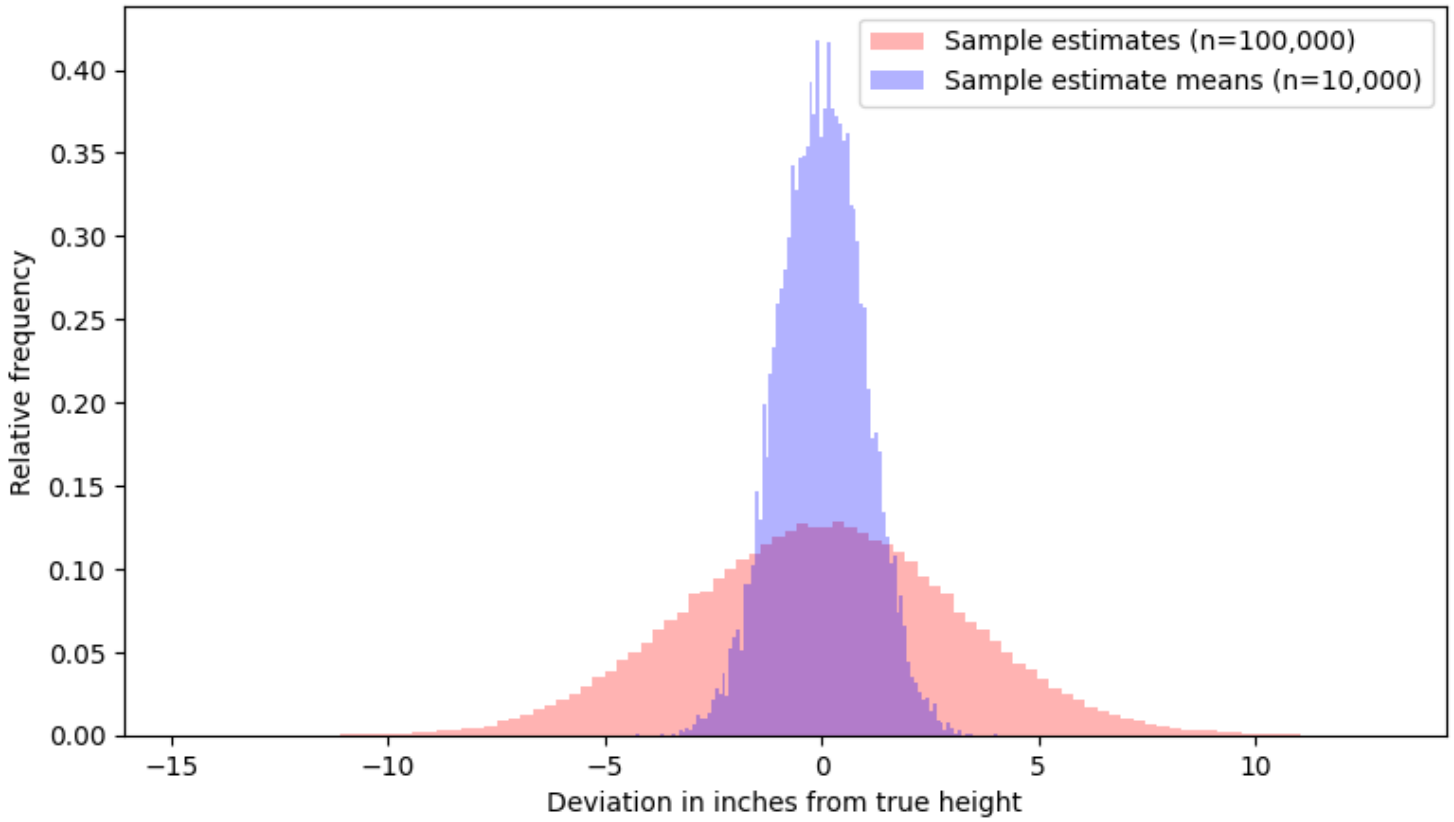
Wisdom of the Crowd: Coin Toss

Consider a slightly biased coin that comes up 51% ”heads” on average.

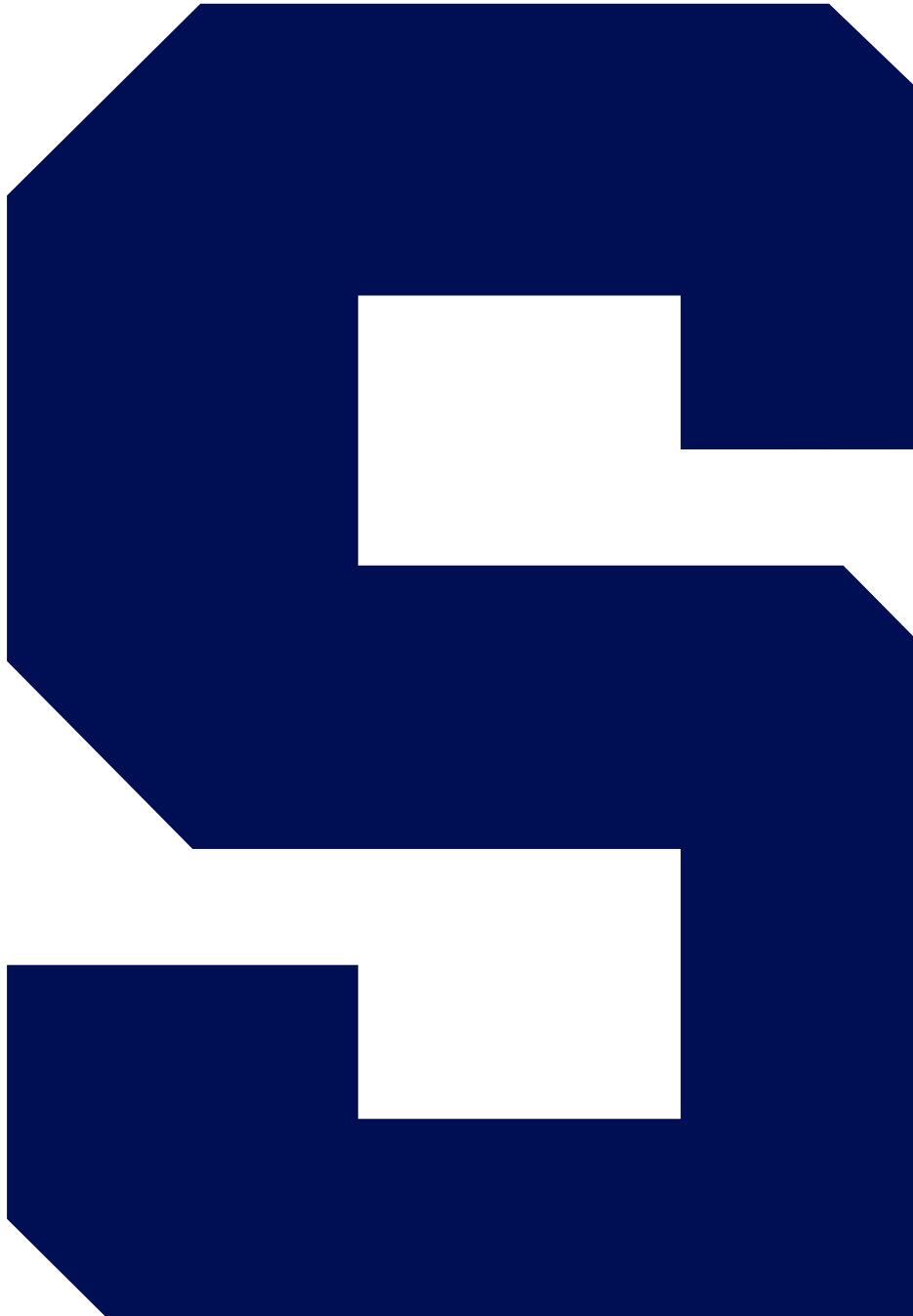


Wisdom of the Crowd: Guessing Height

Ask a large group of individuals to guess someone's height. If guesses are normally distributed with variance V, the average of B guesses will have variance V/B. Example with variance of 10 inches.



Ensemble Methods: Basic Ensemble Techniques



Voting Classifiers

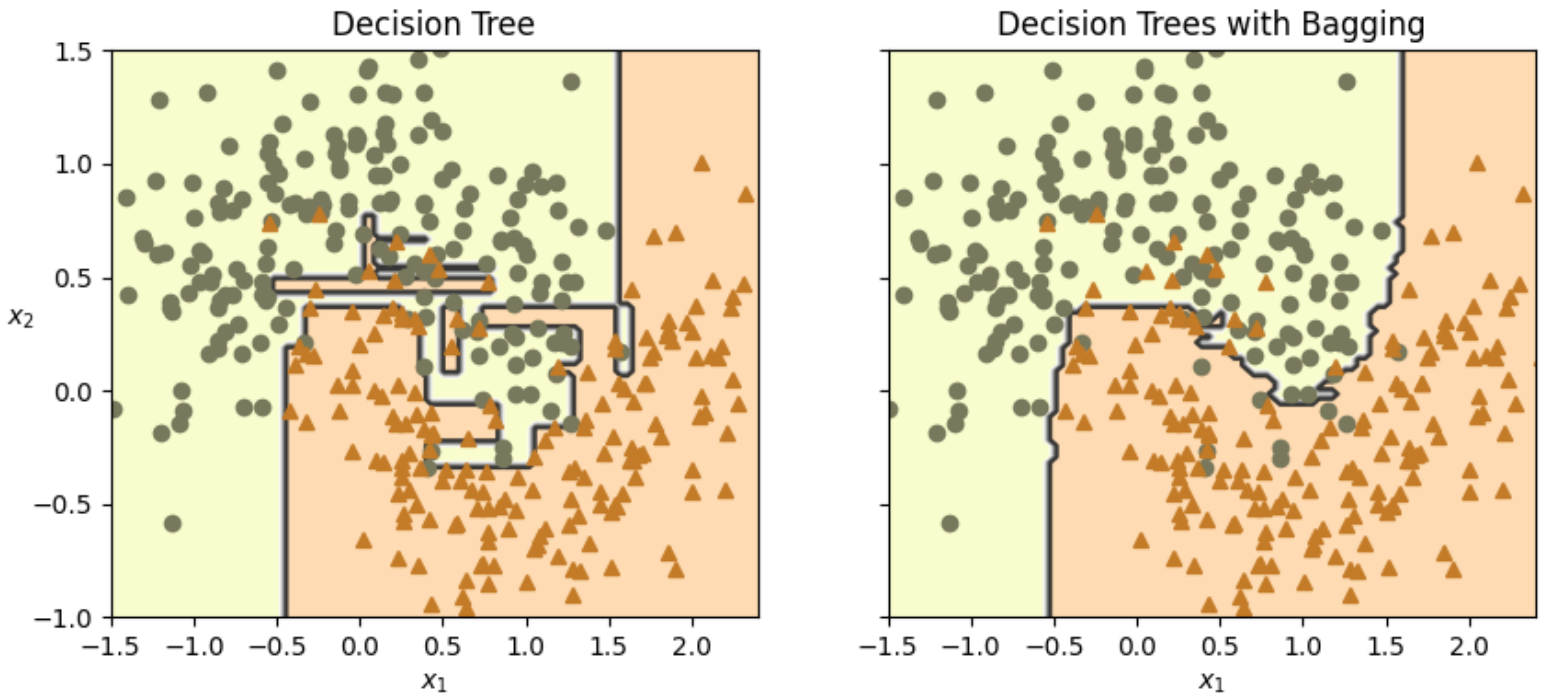
The idea behind the Voting Classifier is to combine conceptually different machine learning classifiers and use either the majority (hard vote) or the average predicted probabilities (soft vote) to predict the class labels.

- **Hard Voting**: Each individual classifier in the ensemble "votes" for a class, and the class that gets most votes is the prediction of the ensemble.
- **Soft Voting**: Every individual classifier provides a probability estimate for each class. The class with the highest average probability across the classifiers is chosen as the prediction.

Soft voting often achieves better results because it gives more weight to highly confident votes.

Bagging (Bootstrap Aggregating)

Bootstrap Aggregating (bagging) is an ensemble method that aims to **reduce the variance** of an estimator by leveraging multiple instances of it.



Bagging Principles of Operation

- Training instances are randomly sampled with replacement. This method of sampling is termed "**bootstrapping**."
- Every predictor in the ensemble is trained on a different bootstrap sample. This means that different instances of the predictor might be seeing slightly different data during their training.
- Once the predictors are trained, the ensemble can make predictions for a new instance by simply aggregating the predictions of all predictors.
  - The aggregation function is typically the **statistical mode** for **classification** or the **average** for **regression**.

Bagging Key Features

- **Inherent parallelism**: Individual predictors can be trained in parallel, as they are independent of each other.
- **Reduction in variance**: As individual predictors might overfit to their specific bootstrap samples, when predictions from all the predictors are aggregated, the ensemble can generalize better and produce a more stable and accurate result.
- **Maintains bias**: The bias remains similar to that of a single predictor because each one is trained on data sampled from the entire dataset.

Random Forests

Random Forests utilize decision trees with bagging. In addition to bootstrap samples, the random forest algorithm introduces **randomness in feature selection** when splitting a node.

Instead of considering all features for a split, a random subset of features is chosen.

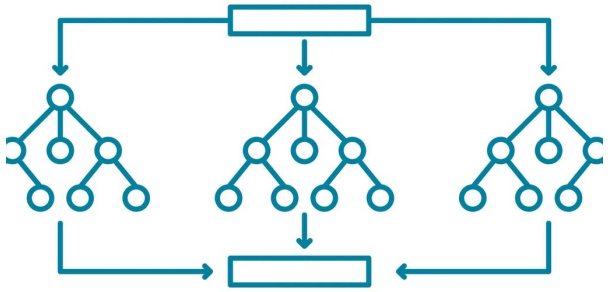
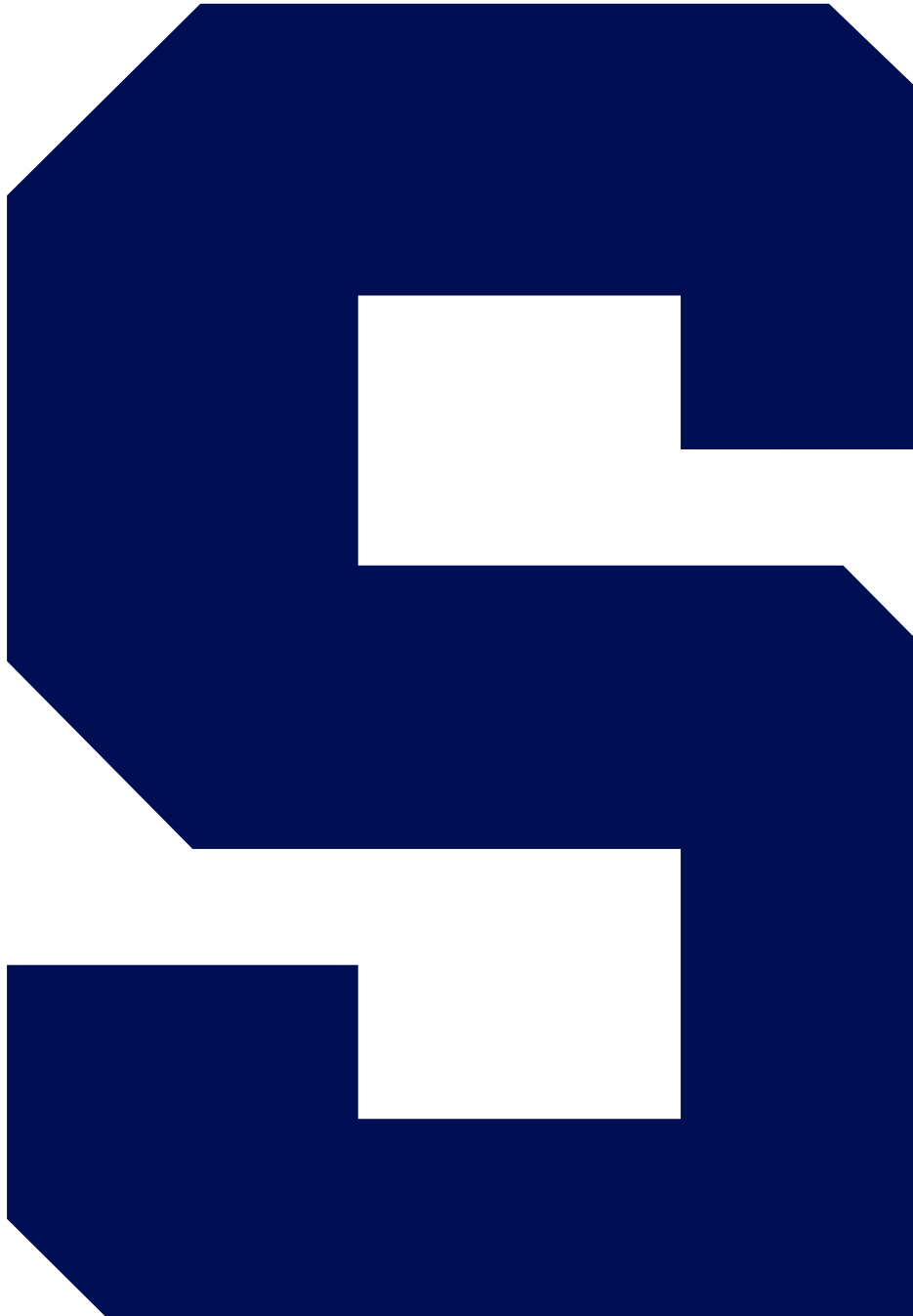


Image source: https://www.turing.com/kb/random-forest-algorithm

Random Forests: Motivations

1. **Decorrelation of trees**: One of the primary reasons for introducing randomness in feature selection at each node is to ensure that individual decision trees in the ensemble are not highly correlated. If we always chose the best features for splitting, many of the trees in the ensemble would look similar (especially at the top splits). This would diminish the benefits of ensemble averaging.
2. **Reduce overfitting**: By considering only a subset of features, we prevent the trees from always splitting on the most dominant features, which can lead to overfitting. Instead, the model is forced to consider other features, leading to more diverse and generalized decision boundaries.
3. **Computational efficiency**: Evaluating the best split is computationally costly. By considering only a subset of features, the training time for each tree is reduced. Given that Random Forests involve training many trees, this computational saving is significant.

Ensemble Methods: Boosting



Principles of Boosting

Boosting, in essence, is about converting **weak learners into strong learners**. Here's how it achieves this:

- **Sequential training:** Unlike Bagging methods which involve parallel training, Boosting focuses on sequential training where each model is trained based on the errors of its predecessors.
- **Error correction:** After training a model, Boosting increases the weight of the misclassified instances. This means that the next model in the sequence is more focused on correctly predicting these instances.
- **Final prediction:** Once all of the predictors are trained, an aggregate prediction is made based on the performance of individual predictors.

Boosting Algorithms: AdaBoost

- The **AdaBoost** (“**adaptive boosting**”) algorithm uses error rates to **increase the weights of misclassified instances** after each training round, making the next classifier more focused on those instances.
- Each predictor is assigned a weight based on its accuracy (lower error rate).
- Predictors that are more accurate have higher weights than less accurate predictors.
- For **classification** problems, the final prediction is made through a **weighted vote**, where each predictor casts as weighted vote for a class, and the class with the highest total weight across all predictors is chosen as the final prediction.
- For **regression**, the predictions of individual predictors can be combined using a **weighted average** based on the predictor weights.

Boosting Algorithms: Gradient Boosting

- Unlike AdaBoost, which increases the weights of misclassified instances, **Gradient Boosting** tries to fit each new predictor in the ensemble to the **residual errors** made by the previous predictor.
- Gradient boosting works particularly well on regression problems, though it can be adapted for classification as well.
- Gradient boosting works best with **"weak learners"** (e.g., a shallow decision tree).

The Gradient Boosting Process

- After initialization (e.g., starting with a mean for regression), the GB algorithm:
    1. Computes the pseudo-residuals (negative gradient) for each instance.
    2. Fits a weak learner to those residuals (not the original target!)
    3. Computes a multiplier for the tree that minimizes loss when added to the ensemble.
    4. Updates the ensemble to include the latest predictor (weighted).
- Steps 1-4 are repeated for M trees to create the final ensemble.
- The final prediction for a given instance is the sum of the initial prediction and the weighted predictions of all the trees.

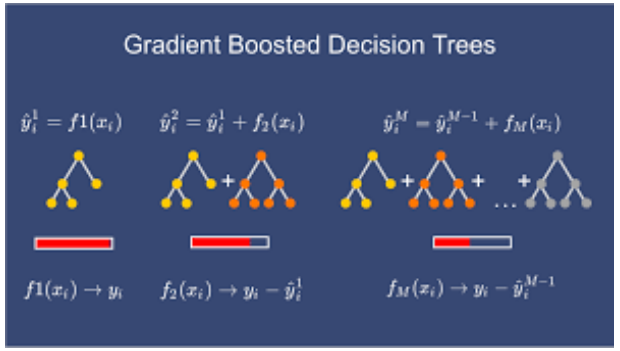


Image sourcehttps://www.linkedin.com/pulse/mastering-gradient-boosting-machine-learning-guide-pratik-thorat/

Bagging Vs. Boosting

Bagging: Create multiple subsets of the original dataset using bootstrap sampling and train a base learner on each subset. Combine their outputs for the final prediction.

Examples: Bagged decision trees, Random Forests.

Advantages:

- **Reduces variance**: Especially useful for algorithms that are sensitive to the specific data they are trained on (like decision trees). By averaging out the results, bagging helps to reduce overfitting.
- **Parallelism**: Each model is trained independently, allowing for parallel processing.
- **Out-of-Bag evaluation**: Since each sample might not see some of the training data (around 37% on average), this unused data can be used for validation, called out-of-bag evaluation.

Boosting: Train models sequentially, with each model trying to correct the mistakes of its predecessor.

Examples: AdaBoost, Gradient Boosting, XGBoost.

Advantages:

- **Reduces bias and variance**: Boosting can capture complex non-linear patterns. By focusing on mistakes, it often results in higher accuracy than bagging.
- **Weighted training**: Boosting assigns weights to training instances, placing more emphasis on those that were misclassified.
- **Model simplicity**: Typically, weak learners (like shallow trees) are used, which are fast to train.

Bagging Vs. Boosting: Which to Choose

- **Nature of data & problem:** If your model is overfitting, bagging is often a good choice. If it's underfitting, boosting might be more suitable.
- **Computational resources:** Bagging can exploit parallelism, so if you have the resources, it can be faster than sequential boosting.
- **Model complexity:** Boosting might result in a more complex model. If interpretability is essential, consider this trade-off.
- **Performance:** Often, in practice, it's beneficial to try both and validate based on performance metrics suitable for the problem at hand.